服务器部署 LLM 配置与输出速度计算公式

01. 简单计算LLM推理和训练所需资源

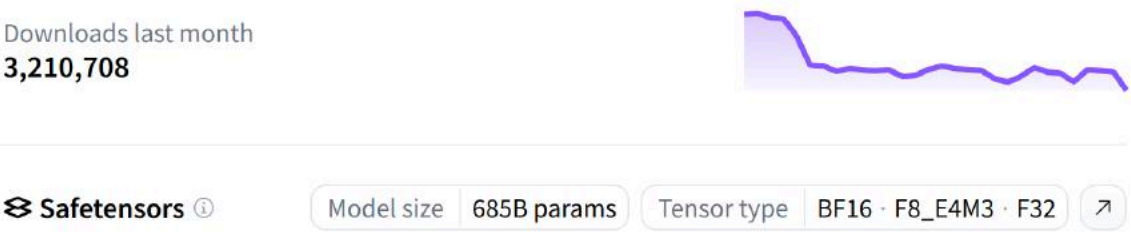
随着开源大模型的性能越来越强，越来越多的场景需要私有化部署大语言模型，不同于商业模式的服务调用，私有化部署需要自己搭建环境，部署模型，有时候还需要自己微调模型，以适应特有的场景。

与其他应用不一样，LLM 的特点是计算要求非常高，一般都是数十亿个参数起步，并且需要对数 TB 的数据进行训练，能合理评估 LLM 推理和训练所需的 GPU 资源也是 AI/LLM 应用开发工程技能的加分项，所以这节课会以一个尽可能简单的方式来总结部署本地 LLM 所需的 GPU 资源，并分享大致的计算原理（仅介绍在本地运行 LLM 的基本内存要求以及基本的优化）。

> 首先是最简单的 GPU 显存估算，推理一般是 `参数数量` * `精度(通常为2或4字节)`，训练一般是 `推理` * `4~6倍` 资源。

例如一个大模型的参数是 1B，则为 10 亿个参数将占据 20 亿字节（10 亿字节等于 1GB），约为 2GB 的显存，32B 的模型就需要占用 64GB 的显存（这是一个近似值，因为1 KB不等于1,000字节，而是1,024字节，可以通过这种简单的方法可以大概评估显存的占用）。

一般在 Huggingface 或者 Ollama 上都可以看到关于模型参数量的说明，例如下图示例：



模型参数为 `685B`，张量类型支持 `BF16(2字节)`、`F8_E4M3(1字节)`、`F32(4字节)`，如果使用 BF16，可以估算加载模型大致的显存为 685B * 2字节 ≈ 1370GB，按单张 H100 的显存为 80G，总共需要需要接近 18 张 H100 才可以加载该模型（deepseek-r1:685B）。

对于**推理**来说，需要资源加载模型权重(参数)并存储 KV 缓存和激活内存，而模型大小是这个公式里比重最大的，所以可以大致估算：

```
总推理内存 = 模型大小 + KV缓存 + 激活内存
```

首先是 `模型大小`，刚刚我们已经讲解了，也就是加载模型所需的显存大小，其中一种常见的优化方式就是**量化**，量化包括以较低的精度加载模型权重。虽然它会影响性能，但效果并不明显，而且比选择具有更高精度的较小模型更可取。

一个 32B 的模型，在 32 位全精度下，每个参数占据 4 字节，需要 128G 显存才可以将该模型加载到 GPU 中，通过将模型权重从 32 位全精度量化为 16 位或者 8 位精度，可以缩减显存到 64GB，甚至 32GB，这时候消费级的显卡也可以运行数百亿参数的大模型。

精度计算公式如下：

- 4 个字节： FP32 / 全精度 / float32 / 32 位
- 2 个字节： FP16 / float16 / bfloat16 / 16 位
- 1 字节： int8/8 位
- 0.5 字节： int4/4 位

总推理内存的第二个指标是 `KV缓存`，在 Transformer 中，解码阶段会在每个时间步骤生成一个 token，具体取决于之前的 token 张量（tensors）。为了避免重新计算这些张量，它们被缓存在 GPU 内存中。

KV 缓存的公式也非常简单：

```
KV缓存 = 2 * 批次大小 * 序列长度 * 层数 * 隐藏层大小 * 精度
```

批次大小、序列长度 、层数 、隐藏大小、精度这些参数听起来比较绕口，如果没有预训练的经验理解起来比较困难，不过只要记住，这些参数也会参与到推理显存的计算，并且这里的大部分参数都可以在模型的信息/说明文档中找到。

例如下方是 `QwQ-32B` 模型对应的相关信息：

This repo contains the QwQ 32B model, which has the following features:

- Type: Causal Language Models
- Training Stage: Pretraining & Post-training (Supervised Finetuning and Reinforcement Learning)
- Architecture: transformers with RoPE, SwiGLU, RMSNorm, and Attention QKV bias
- Number of Parameters: 32.5B
- Number of Paramaters (Non-Embedding): 31.0B
- Number of Layers: 64
- Number of Attention Heads (GQA): 40 for Q and 8 for KV
- Context Length: Full 131,072 tokens
    - For prompts exceeding 8,192 tokens in length, you must enable YaRN as outlined in this section.

总推理内存的第三个指标是 `激活内存`，在模型的前向传递过程中，会存储中间激活值。这些激活值表示数据在模型中向前传播时神经网络中每一层的输出，并且数据保存在 FP32 中，以避免数值爆炸并确保收敛。

```
激活内存 = 批次大小 * 序列长度 * 隐藏大小 * (34 + (5 * 序列长度 * 注意力头数量) / (隐藏大小))
```

通过上述的三个指标，我们可以得出单一模型所需的总推理内存，并且可以很容易观察到后两个指标对比第一个指标，在数据上会小的多，所以可以通过 `模型大小` 大致来估量所需的显存。

对于并发调用的情况，需要同时运行多个模型实例，那么总的内存需求将是单个模型层的内存需求乘以并发调用的数量。这是因为每个并发调用都需要独立的模型实例来处理，而这些实例将共享或占用相同的内存资源。

对于**训练场景**而言，由于添加了优化器和梯度，所需的资源比**推理**要多很多，计算公式如下：

```
总内存 = 模型大小 + KV 缓存 + 激活 + （优化器状态 + 梯度）* 可训练参数数量
```

组件类型	每个参数的字节数
模型参数（权重）	每个参数 4 字节

组件类型	每个参数的字节数
Adam 优化器（2个状态）	每个参数 8 字节
梯度	每个参数 4 字节
激活和临时内存（可变大小）	每个参数 8 字节

而附加的组件导致每个模型的参数需要大约 12~20 字节的额外 GPU 内存，简化公式如下：

```
训练总内存 ≈ 模型大小 + (12 ~ 20字节) * 可训练参数数量
          ≈ 模型大小 + (3 ~ 5) * (4字节 *模型参数)
          ≈ 模型大小 + (3 ~ 5) * 模型大小
          ≈ (4 ~ 6倍) * 模型大小
          ≈ (4 ~ 6倍) * 推理内存
```

至此我们就可以大概估算出部署一个 LLM 大概需要多少显存才能正常运行，当然实际因为各种部署的优化（vLLM），实际的数值可能会适当减少或者增加，如果觉得计算过程不好记忆，也可以考虑使用这个在线计算工具：https://llm-dev-tools.streamlit.app/Memory

02. LLM 输出速度计算公式

在部署本地 LLM 的时候，除了需要计算部署所需的 GPU 显存外，另外一个很重要的指标就是 `大模型输出速度`，即 `生成Token的速度`，目前该指标被拆分成两个阶段进行计算：

1. Prefill：预填充，并行处理输入的 tokens（生成第一个 token 所花的时间）；
2. Decoding：解码，逐个生成下一个 token（后续每个 token 所需的时间）；

除此之外，在很多公司内部还可以经常看到这类术语，我们也初步了解下：

1. 首 Token 延迟，Time To First Token (**TTFT**)，Prefill，Prefilling，指的都是从输入到输出第一个 token 的延迟；
2. 每个输出 token 的延迟（不含首个Token）,Time Per Output Token (**TPOT**)，指的是第二个token开始的吐出速度；
3. 延迟 **Lantency**，理论上即从输入到输出最后一个 token 的时间，计算公式：Latency = TTFT + TPOT * 生成 token 数；
4. 每秒生成 Token 数 (**TPS**)，计算公式：生成总 Token 数 / 延迟；

可以发现，上述无论哪种表达方式，第一个 token 的生成和剩余 tokens 的生成是不一样的，这是因为 LLM 推理加速原因导致的。要计算 LLM 输出 Token 的速度，我们需要了解几个计算量：

1. `FLOPS`：全称 Float Point Operations pre Second，即每秒浮点运算次数，是一个用于评估计算速度的单位，除此之外经常能在论文、GPU 性能上看到这些指标 `MFLOPS(每秒一百万次浮点计算)`、`GFLOPS(每秒十亿次浮点数计算)`、`TFLOPS(每秒一万亿次浮点数计算)`、`PFLOPS(每秒一千万亿次浮点数计算)`、`EFLOPS(每秒一百亿亿次浮点数计算)`。
2. `OPS`：和 `FLOPS` 一样也是一个用来评估计算速度的单位，不过 `FLOPS` 针对的是浮点数，`OPS` 针对的是整型，例如 `TOPS` 为万亿次操作/每秒，该指标一般用于 AI 推理/计算，因为 AI 计算主要依赖于低精度整数（如 INT8、INT4）运算，以提高计算效率和能效，而训练等任务通常需要高精度（如 FP32、FP64）浮点运算。

一些常见的显卡算力如下：

型号	算力	显存	带宽	功耗
昆仑芯 R200	FP32: 32 TOPS FP16: 128TOPS INT8: 256 TOPS	16GB	512GB/s	150W
H800	FP32: 67 FTOPS FP16: 1979 FTOPS INT8: 3958 TOPS	80GB HBM3	2TB/s	300-350W
V100	FP32: 15.7 FTOPS FP16: 125 FTOPS INT8: 250 TOPS	16GB HBM2	900GB/s	300W
A100	FP32: 19.5 FTOPS FP16: 312 FTOPS INT8: 624 TOPS	40GB HBM2	1.6TB/s	400W
A10	FP32: 31.25 FTOPS FP16: 125/250 FTOPS INT8: 250/500 TOPS	24GB GDDR6	600GB/s	150W
TPU V4 TPU V3 TPU V2	INT8: 275 TOPS INT8: 123 TOPS INT8: 32TOPS	32 GB	6.4TB/s	262W
P4	FP32: 5.5 FTOPS FP16: 22.1 FTOPS INT8: 22.1 TOPS	8GB GDDR5	192GB/s	75W
T4	FP32: 8.1 FTOPS FP16: 65 FTOPS INT8: 130 TOPS	16GB GDDR5	320GB/s	70W

有了这些信息后，就可以计算理论情况下的大模型推理速度了，推理过程的主要计算量在 Transformer 解码层，这一层对每个 Token、每个模型参数都是一个单位的计算量，所以在推理过程中，每一个 Token/模型参数，需要进行 1单位×2 flops = 2 次浮点数运算。

基于这个理论，我们可以得到针对不同 prompt，模型的推理量计算公式如下：

$$\text{FLOPS of Inference} = 2N\,\text{FLOPS/Token} \times \text{Prompt Tokens}$$

即推理总计算量 = 2 * 模型参数 * 提示词token数，例如模型参数为 32B，提示词token数为 1000，则总计算量为 6.4E+13，是一个非常庞大的计算量。

如果还想评估每秒输出的 Token 数，还需要考虑 GPU 利用率，公式如下：

$$\text{GPU of Inference} = \frac{2N\,\text{FLOPS/Token} \times \text{Prompt Tokens}}{\text{Peak Flops} \times \text{Flops Utilization}}$$

例如我们使用 10 张 A100 显卡，单卡有 312 TFLOPS 的算力，GPU 的利用率为 46.2%（一般查询模型的信息或者评测可以看到），如果使用一个 32B 的模型，BF16 精度，prompt 平均长度为 1K。则平均输出速度计算如下：

$$\text{平均速度} = \frac{2 * 32B\,\text{FLOPS/Token} \times 1\text{K Prompt Tokens}}{10 * 312\ \text{TFLOPS} \times 46.2\%}$$

计算得到结果为 0.0444s/Token，平均一秒钟生成的 Token 数为 22.5225 个，非常慢是不是，所以在实际的部署中，一般会使用 `kv缓存` 来提升推理的速度，KV Cache 采用以空间换时间的思想，复用上次推理的 KV 缓存，可以极大降低内存压力、提高推理性能，而且不影响任何计算精度。

在这种架构下，生成第一个token的时候，kv是没有任何缓存的，需要预填充prompt对应的KV矩阵做缓存，所以第一个token生成的最慢，而从第二个token开始，都会快速获取缓存，并将前一个token的kv也缓存，这是一个空间换时间的方案，关于这里细节的部分就不继续深入探讨了，涉及到 LLM 算法/架构层面的知识，如果感兴趣可以自行搜索 or 向 ChatGPT 提问。